

You are analyzing an AliBee Gemini processing debug log stored in JSONL format.

File format

The file uses JSON Lines format:

- Each line is an independent JSON object.
- Do not treat the entire file as one JSON array.
- Parse and analyze the file line by line.
- Some fields may contain long raw Gemini responses with multilingual content.
- Do not skip malformed lines silently. Report their line numbers and parsing errors.

Purpose of the log

The log records the complete lifecycle of products processed by a Gemini-based product enrichment worker.

It is designed to show:

- Which products were processed.
- What Gemini returned.
- Whether the Gemini response was valid JSON.
- Whether a safe JSON repair was applied.
- Whether all required validation checks passed.
- Whether the result was saved to the database.
- Whether `ai_processed_at` was updated.
- Whether a product was blocked because of `PROHIBITED_CONTENT`.
- Whether `ai_failed` was set to `1`.
- API errors, temporary failures, and rate-limit events.
- Token usage and image information.
- Final run-level statistics.

Important fields

Depending on the event type, a record may include fields such as:

- `record_type`
- `timestamp`
- `product_id`
- `product_title`
- `status`
- `raw_response`
- `parsed_json_written_to_db`
- `json_error`
- `validation_error`
- `json_repair`

- `images_requested`
- `images_sent`
- `image_urls`
- `prompt_tokens`
- `output_tokens`
- `total_tokens`
- `db_rows_saved`
- `ai_processed_at_updated`
- `ai_failed_value`
- `error`
- `error_code`
- `block_reason`

The exact fields may vary between records.

Possible product statuses

Interpret statuses carefully. Common examples include:

- `success_db_saved`
 - The Gemini response passed parsing and validation.
 - The database write completed.
- `ai_processed_at` should normally be updated.
- `invalid_json_or_validation_failure`
 - The response could not be parsed as valid JSON, or it failed required validation.
 - It should not be considered successfully saved.
- `prohibited_content`
 - Gemini blocked the request because of prohibited content.
 - Confirm that `ai_failed` was updated to `1`.
 - Confirm that `ai_processed_at` remained unchanged or false.
- `rate_limit_or_temporary_api_error`
 - The request failed because of a temporary API, network, rate-limit, or model availability problem.
- `temporary_skip`
 - The product was skipped temporarily, for example because one or more images could not be downloaded.

The file may contain additional statuses. List and explain every unique status found.

Safe JSON repair logic

The script may apply a narrowly limited JSON repair.

The allowed repair is:

- Gemini returned one complete, balanced root JSON object.
- After that complete object, the response contained only whitespace and one or more extra closing braces: `}`.
- The script removed only those trailing extra closing braces.

The repair must not:

- Complete truncated JSON.
- Add missing fields.
- Add missing braces inside the root object.
- Guess missing text.
- Extract a nested language object instead of the root object.
- Ignore explanatory text after the JSON.
- Modify generated values.

A repaired response is trustworthy only when all of the following are true:

1. `json_repair.applied` is true.
2. The repair type indicates removal of trailing extra closing braces.
3. The removed text contains only whitespace and `}` characters.
4. The repaired JSON parsed successfully.
5. The root `product_id` matches the processed product ID.
6. All mandatory validation checks passed.
7. The database write completed successfully.

Mandatory content validation

For each product reported as successfully saved, verify that the parsed object contains all six required languages:

- `english`
- `hebrew`
- `arabic`
- `french`
- `spanish`
- `ruddian`

For each language, verify:

- A non-empty `name`.
- A non-empty `description`.
- Exactly 10 hashtags.
- `reviews` is an array.

Also verify the required product-level fields, where present:

- `product_id`
- `detected_product_type`
- `first_category_id`
- `first_category_name`
- `second_category_id`
- `second_category_name`
- `third_category_id`
- `third_category_name`
- `category_path`
- `category_confidence`
- `category_reason`
- `confidence`
- `image_score`
- `content_sensitivity_level`
- `audience_gender_code`
- `is_giftable`
- `product_insights`

Verify that `product_insights` contains the expected nested sections:

- `target_audience`
- `purchase_intent`
- `marketing_angles`
- `product_attributes`

Report any successfully saved product that appears to be missing required data.

Database consistency checks

For every `success_db_saved` record, check:

- `db_rows_saved` has a successful value, normally 6 language rows.
- `ai_processed_at_updated` is true.
- `ai_failed_value` is not `1`.
- `parsed_json_written_to_db` exists.
- The `product_id` in `parsed_json_written_to_db` matches the record's `product_id`.

For every `prohibited_content` record, check:

- `ai_failed_value` is `1`.
- `ai_processed_at_updated` is false or not updated.
- The block reason is actually `PROHIBITED_CONTENT`.
- The product was not also marked as successfully saved.

Report contradictory states, for example:

- `status` says success but `db_rows_saved` is zero.
- `status` says failure but `ai_processed_at_updated` is true.
- `PROHIBITED_CONTENT` occurred but `ai_failed` was not set to `1`.
- `ai_failed=1` was set for a reason other than `PROHIBITED_CONTENT`.
- The product ID in the Gemini JSON differs from the processed product ID.

JSON repair analysis

Create a dedicated section for JSON repairs.

For every repaired product, report:

- Product ID.
- Product title.
- Repair type.
- Exact removed text in an escaped representation.
- Number of removed characters.
- Whether parsing succeeded.
- Whether mandatory validation passed.
- Whether the database save succeeded.
- Whether `ai_processed_at` was updated.
- Whether the repaired JSON appears complete.

Inspect the end of each `raw_response` and confirm whether it looks like:

```
}  
}
```

or another sequence where a complete root JSON object was followed only by extra closing braces.

Flag the repair as unsafe or suspicious if:

- The raw response appears truncated.
- It ends inside a string, array, or nested object.
- Required languages or fields are absent.
- Text other than whitespace and `}` followed the root object.
- The parsed product ID does not match.

- Validation failed.
- The repaired result was saved despite failed validation.

Detecting truncated output

Look for evidence of a response being cut off:

- Unexpected end of JSON input.
- Unterminated strings.
- Missing closing arrays or objects.
- A raw response ending in the middle of a sentence or field.
- Missing required languages near the end of the expected schema.
- Output token count reaching or approaching the configured maximum.
- Finish reasons indicating token exhaustion.

Do not classify a trailing extra brace as truncation when a complete and validated root object exists before it.

API and operational errors

Group and count errors by category:

- PROHIBITED_CONTENT
- Invalid JSON
- Validation failure
- HTTP 429 or quota errors
- HTTP 503 or model high demand
- Network errors
- Database errors
- Image download failures
- Temporary skips
- Unknown errors

List the affected product IDs for each category.

Distinguish temporary infrastructure errors from permanent product-level failures.

Image checks

For each product, compare:

- Number of image URLs available.
- Number of images requested.
- Number of images successfully sent.

Report:

- Products with fewer images sent than requested.
- Products with zero images.
- Products skipped because image downloads failed.
- Whether a smaller image count was legitimate because fewer URLs were available.

Do not report a product as having an image failure merely because it had fewer than six images when only that number of image URLs existed.

Token analysis

Summarize:

- Minimum, maximum, average, and median prompt tokens.
- Minimum, maximum, average, and median output tokens.
- Minimum, maximum, average, and median total tokens.
- Products with unusually high token usage.
- Whether invalid or repaired JSON correlates with high output-token usage.
- Whether any response appears to have reached the configured output-token limit.

Duplicate and consistency checks

Check for:

- Duplicate product IDs.
- A product recorded as both failed and successful.
- Multiple database saves for the same product.
- Multiple different raw responses for the same product.
- Run counters that do not match the number of product records.
- Missing start or end records.
- Records with missing timestamps.
- Records appearing out of chronological order.

Run summary

If the file contains a `run_end` record, compare its totals with independently calculated totals.

Verify values such as:

- Total product attempts.
- Successfully saved products.
- Failed products.
- Temporarily skipped products.
- `PROHIBITED_CONTENT` products.
- JSON repairs detected.

- JSON repairs parsed successfully.
- JSON repairs that passed validation.
- JSON repairs saved to the database.
- JSON repairs rejected.

Do not trust the summary blindly. Recalculate totals from the product records and report any mismatch.

Required final report

Produce a structured report with these sections:

1. Executive summary

Include:

- Number of products processed.
- Number successfully saved.
- Number failed.
- Number temporarily skipped.
- Number blocked by `PROHIBITED_CONTENT`.
- Number of JSON repairs.
- Whether the run appears healthy.

2. JSON repair review

Provide a table with:

- Product ID
- Repair applied
- Characters removed
- Parse succeeded
- Validation passed
- Database saved
- Repair assessment

Use assessment values:

- Safe
- Suspicious
- Unsafe

3. Failures and warnings

Group failures by reason and list affected product IDs.

Separate actual failures from warnings that did not prevent a database save.

4. Database integrity review

Report contradictions, missing fields, mismatched IDs, or suspicious writes.

5. Image and token review

Summarize image delivery and token usage anomalies.

6. Final conclusion

Clearly answer:

- Were the JSON repairs safe?
- Did any incomplete JSON enter the database?
- Did every repaired result pass validation?
- Were any products incorrectly marked as processed?
- Were any `PROHIBITED_CONTENT` products incorrectly handled?
- Is the script safe to continue running in its current form?

Be precise and evidence-based. Quote short relevant snippets from the log when useful, but do not reproduce complete large Gemini responses.